

Part B: Genetic Algorithm

Word Count: 1,992

1. Introduction

In this project, a simulated creature in PyBullet evolved using a Genetic Algorithm (GA) to be able to move and climb a mountain environment. Every creature's body structure and joint function are determined by its encoded genome. The encoding scheme and the GA parameters were changed in experiments to compare their effects on final fitness and learning speed. Performance is evaluated using a fitness function that avoids actions that do not show true climbing while rewarding steady improvement and height gain on the mountain.

2. Integration

2.1 Initial Integration

cw-envt.py defined the mountain-climbing environment. All creature evaluations and CSV replays were carried out in Gaussian pyramid mountain arena. This ensure the GA was choosing behaviours for the purpose being performed movement and climbing on the mountain.

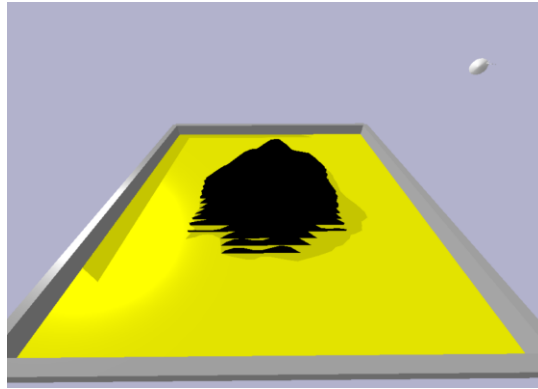
```
# Integrate the mountain arena instead of the floor
arena_size = 20
cw_envt.make_arena(arena_size=arena_size)

mountain_position = [0, 0, -1]
mountain_orientation = p.getQuaternionFromEuler([0, 0, 0])
p.setAdditionalSearchPath("shapes", physicsClientId=pid)
p.loadURDF("gaussian_pyramid.urdf", mountain_position,
          mountain_orientation,
          useFixedBase=1,
          physicsClientId=pid)
```

Code Snippet 1: Integration of the mountain (simulation.py)

```
# Mountain arena was integrated by me
arena_size = 20
cw_envt.make_arena(arena_size=arena_size)
mountain_position = [0, 0, -1]
mountain_orientation = p.getQuaternionFromEuler([0, 0, 0])
p.setAdditionalSearchPath("shapes")
# can change and load the different landscapes here
p.loadURDF(
    "gaussian_pyramid.urdf",
    mountain_position,
    mountain_orientation,
    useFixedBase=1
)
```

Code Snippet 2: Integration of the mountain (realtime_from_csv.py)



Img 1: pybullet integration of mountain terrain

After the simulation started, the creature was seen to launch upward and leave the sandbox area. Anti-cheat constraints were put in place to detect and stop evaluations in which the creature crosses the arena boundaries or stays in the air for a long time. These limitations ensure real climbing performance is rewarded by the GA and that replay behaviour follows to the same rules.

2.2 Anti-cheat Functions

2.2.1 Arena Boundary

A creature cannot leave the sandbox due to this check. The creature's base position is compared to the sandbox limits half the arena size in the X and Y directions at each stage of the simulation.

```
# anti-cheat function to stop if creature leaves the arena
if abs(pos[0]) > half or abs(pos[1]) > half:
    cr.out_of_bounds = True
    cr.cheated = True
    break
```

Code Snippet 3: anti cheat arena boundary (simulation.py)

The run is immediately recognised as cheated, and the evaluation stops if the creature crosses the boundary. This keeps the results consistent between runs and ensures that all fitness is gained only within the given sandbox and mountain.

2.2.2 Airborne detection

This check stops creature from cheating by flying because of strong motor impulses or collision launches. The simulation looks for the creature's body's contact points at each step and if the creature makes no contact with any objects, it is in the air.

```

# anti-cheat function is creature flying in the air
flyingair = p.getContactPoints(bodyA=cid, physicsClientId=pid)
airborne = True
for check in flyingair:
    # id of the other body in contact
    body = check[2]
    # ignore the self contact
    if body != cid:
        airborne = False
        break

# airborne steps count
if airborne:
    cr.airbornenumber += 1
else:
    cr.airbornenumber = 0
# if airborne too long, creature cheat
if cr.airbornenumber > 480:
    cr.cheated = True
    break

```

Code snippet 4: anti cheat airborne detection (simulation.py)

The code uses an airborne number counter to track consecutive airborne steps. The run is marked as cheated and stop if the creature stays airborne for more than 480 simulation steps. The airbornenumber resets to 0 when the creature touches the ground within the threshold.

Together, these checks satisfy the no-cheating requirement and keep comparisons fair.

2.3 Start position of the creature

The creature's start point was shifted to a place close to the mountain base to avoid not real launch behaviour at the beginning of the run. This ensures that fitness occurs by contact-based climbing rather of an early jump, reduces initial collision forces, and allows the creature to relax under gravity.

```

# spawn creature near the mountain if not consider as cheated
try:
    p.resetBasePositionAndOrientation(cid, [6.0, 0.0, 2.5], [0, 0, 0, 1], physicsClientId=pid)
    spawn_pos, _ = p.getBasePositionAndOrientation(cid, physicsClientId=pid)
except Exception:
    cr.cheated = True
    return

```

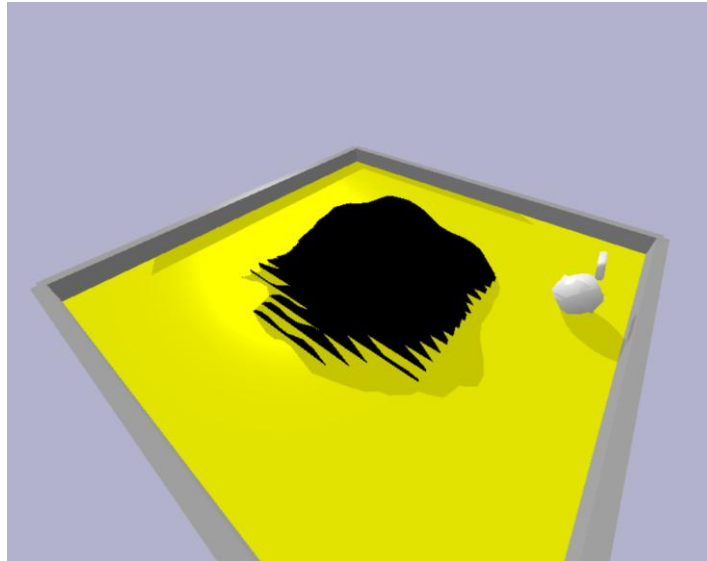
Code Snippet 5: start position changed to be near mountain (simulation.py)

```

# start position changed by me no more air drop
p.resetBasePositionAndOrientation(rob1, [6.0, 0.0, 2.5], [0, 0, 0, 1])
start_pos, orn = p.getBasePositionAndOrientation(rob1)
cr.update_position(start_pos)

```

Code Snippet 6: start position changed to be near mountain (realtime_from_csv.py)



Img 2: Creature starts near the mountain

3. Fitness Function

3.1 Update position

The first starter implementation calculated straight-line movement and just saved start and end positions. That works well for a move anywhere task, but it is not appropriate for mountain climbing because a creature may score highly by launching itself, moving sideways, or moving away from the mountain without having any contact with the slope.

```
def update_position(self, pos):  
    if self.start_position == None:  
        self.start_position = pos  
    else:  
        self.last_position = pos
```

Code snippet 7: initial starter code for the update position (creature.py)

update_position() was changed from basic tracking into a tracker that store several signals that are relevant to increasing during the run.

```

def update_position(self, pos):
    # x, y, z position
    x, y, z = pos

    # start values based on the first position
    if self.start_position is None:
        self.start_position = pos
        self.last_position = pos
        self.prev_pos = pos
        # tracking values
        self.max_height = z
        self.min_radius = float(np.sqrt(x*x + y*y))
        self.maxheightnear = None
        self.uphill_score = 0.0
        self.near_steps = 0
        return

    # update the last position.
    self.last_position = pos

    # radius from center
    radius = float(np.sqrt(x*x + y*y))

    # track the closest distance to center
    if self.min_radius is None or radius < self.min_radius:
        self.min_radius = radius

    # overall max height
    if self.max_height is None or z > self.max_height:
        self.max_height = z

    # creature is near the slope area, this step as near step.
    if radius <= 5.5:
        self.near_steps += 1

        # Tracking the highest height reached while near the mountain
        if self.maxheightnear is None or z > self.maxheightnear:
            self.maxheightnear = z

        # If have a previous position change in height
        if self.prev_pos is not None:
            height = float(z - self.prev_pos[2])
            if height > 0:
                self.uphill_score += height

    # current position as before
    self.prev_pos = pos

```

Code snippet 8: updated update position function (creature.py)

With each step, it calculates the creature's radial distance to the mountain centre and keeps track of the smallest radius achieved, which shows the creature has reached the mountain area. It uses radius more than or equal to 5.5 to track maximum height while close to the mountain, ensuring that height is only recorded

when the creature is really interacting with the mountain area. It collects an uphill score by adding up only positive height changes in the slope, recording continuous climbs instead of a single spike.

3.2 Fitness Function

Only the straight-line distance between the start and end spots was used in the starter fitness. This is acceptable for simple movement, but it does not compare to mountain climbing.

```
def get_distance_travelled(self):  
    if self.start_position is None or self.last_position is None:  
        return 0  
    p1 = np.asarray(self.start_position)  
    p2 = np.asarray(self.last_position)  
    dist = np.linalg.norm(p1-p2)  
    return dist
```

Code snippet 9: Initial fitness function (creature.py)

So, the new fitness function was modified as a weighted climbing score based on run data.

```

def get_distance_travelled(self):
    # run is invalid or missing data, fitness 0
    if self.cheated or self.start_position is None or self.last_position is None:
        return 0.0

    # start position
    sx, sy, sz = self.start_position

    # last position
    lx, ly, lz = self.last_position

    # radius from the centre
    startradius = float(np.sqrt(sx*sx + sy*sy))

    # last radius from the centre
    lastradius = float(np.sqrt(lx*lx + ly*ly))

    # how much closer it moved to the centre
    progress = max(0.0, startradius - lastradius)

    # reward if it got the near slope
    if self.min_radius is None:
        close = 0.0
    else:
        close = max(0.0, 5.5 - float(self.min_radius))

    # best height gained near the mountain
    if self.maxheightnear is None:
        bestheightincrease = 0.0
    else:
        bestheightincrease = max(0.0, float(self.maxheightnear) - float(sz))

    # final height gained
    finalheight = max(0.0, float(lz) - float(sz))

    # continuous climb
    uphill = max(0.0, float(self.uphill_score))

    # rewards weightage
    W_BEST = 100.0
    W_CLOSE = 25.0
    W_UPHILL = 15.0
    W_FINAL = 20.0
    W_PROGRESS = 8.0

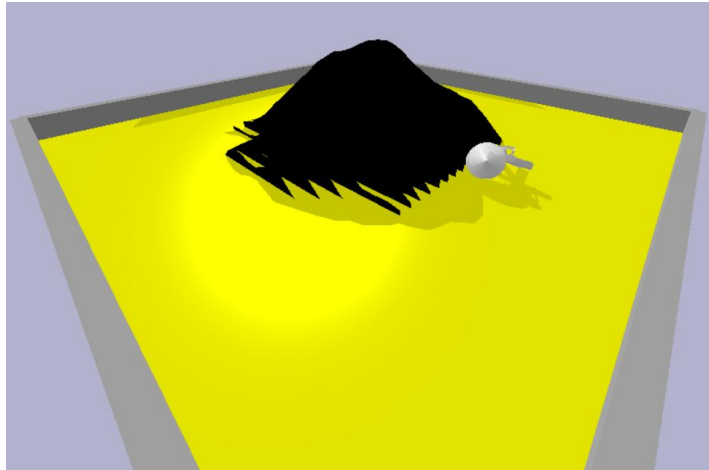
    # fitness score calculation
    fitness = (W_BEST * bestheightincrease + W_FINAL * finalheight + W_UPHILL * uphill + W_PROGRESS * progress + W_CLOSE * close)

    # does not return 0
    return max(1e-6, float(fitness))

```

Code snippet 10: new fitness function with the rewards (creature.py)

For early generations to approach the mountain, it first awards points for getting closer to the mountain. The minimum radius reached, which shows getting close and engaging the slope region, is then used to determine a closeness reward. The best height achieved close to the mountain makes up most of the score since it indicates actual climbing on the slope rather than a height spike elsewhere. It includes final height gain at the end of the duration and constant upward gain to encourage steady climbing.



Img 3: creature reaches mountain and trying to climb

The image confirms that the creature arrives at the mountain and starts climbing. Its posture and movement show steady uphill progress, which is in line with the fitness benefits of being close to the slope and gaining height.

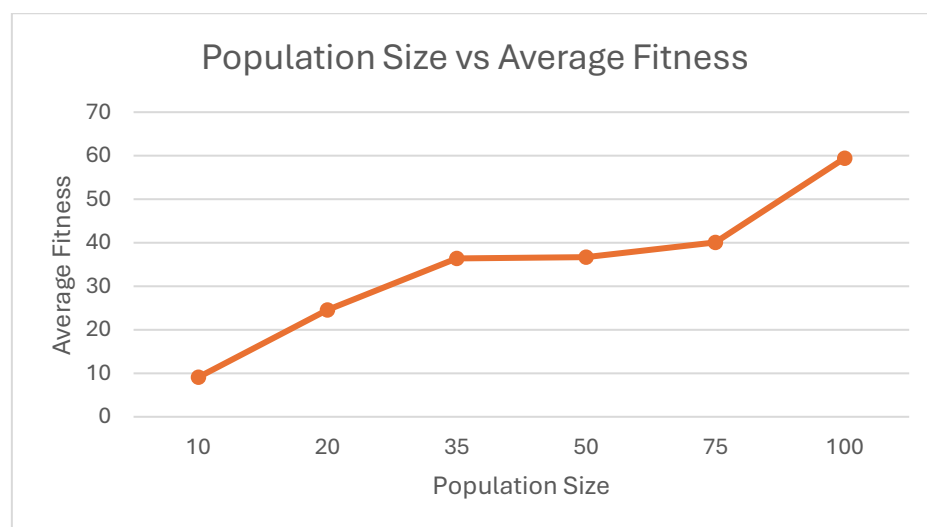
4. Series of Experiments

Three runs of each setting were tested. The maximum level of fitness reached over all runs is known as Best Fitness (Highest). Average Best Fitness is the average of the three runs of final best fitness, while Average Mean Fitness is the average of the three runs of final mean fitness.

4.1 Population Size

4.1.1 Population Size vs Average Fitness

The mean-fitness increases as population size increases, shows that larger populations help GA in maintaining higher overall population quality during training.

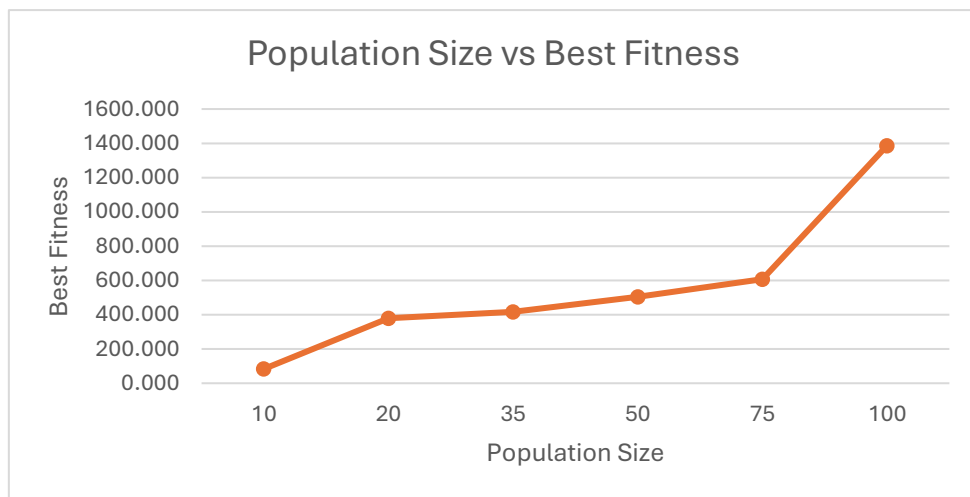


Img 4: Graph for the population size vs average fitness

Larger populations have more options for selection and greater variety, which causes the average performance to increase across generations, while smaller populations have many weak individuals and low mean fitness.

4.1.2 Population Size vs Best Fitness

Because larger populations include more creatures per generation and increase the chance of finding at least one great climber, best fitness usually rises with population size.



Img 5: Graph for the population size vs best fitness

However, because GA results are random and a single outlier can dominate best metric in any one run, best-fitness curves show some huge jumps example for population 75 to 100.

4.1.3 Table Summary

Run ID	Population Size	Initial Genome Size	Mutation Rate	No. of Generation	Simulation Rate	Best Fitness (Highest)	Average mean Fitness (Out of 3 Runs)	Average Best Fitness (Out of 3 Runs)
1	10	3 genes	0.1	300	2400	149.323	9.078	83.534
2	20	3 genes	0.1	300	2400	985.023	24.560	378.484
3	35	3 genes	0.1	300	2400	1058.465	36.408	416.122
4	50	3 genes	0.1	300	2400	689.259	36.678	504.423
5	75	3 genes	0.1	300	2400	1006.88	40.038	606.609
6	100	3 genes	0.1	300	2400	1580.507	59.415	1386.715

Table 1: Population Size

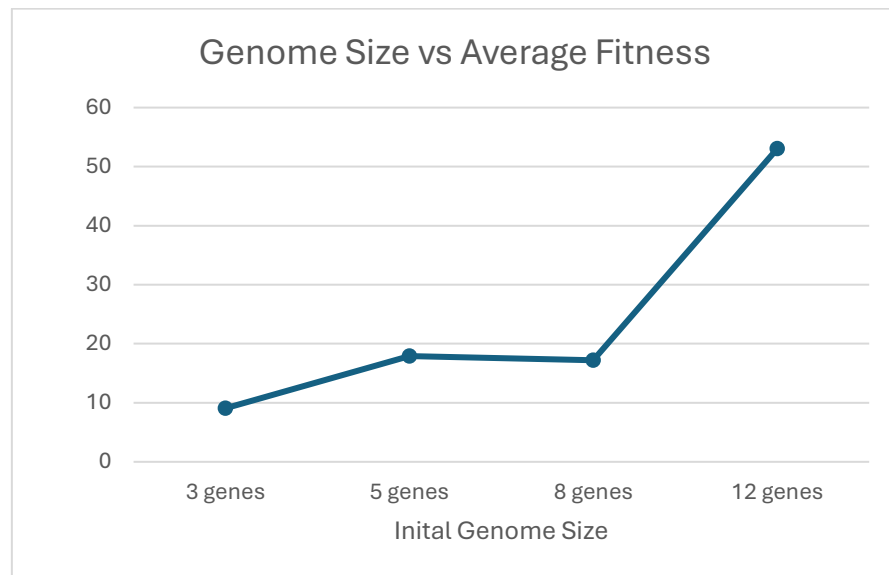
Average Best Fitness rises from 83.534 to 1386.715 and Average Mean Fitness rises from 9.078 to 59.415, showing both improved average behaviour and stronger best solutions as population size increases. The more samples

the higher chance of a top performer result continues with the maximum Best Fitness reaching its highest recorded value at population 100.

4.2 Genome size

4.2.1 Genome Size vs Average Fitness

The mean-fitness plot clearly improves with increasing genome size. Since most creatures do not consistently reach the peak or provide stable climbing, the mean remains low with three genes.

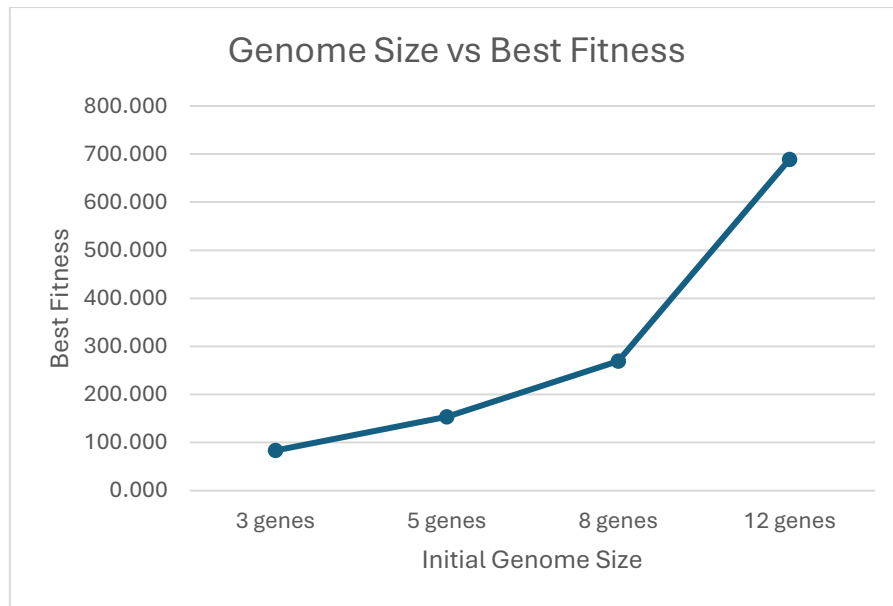


Img 6: Graph for the genome size vs average fitness

The mean rises faster and remains higher with 5 and 8 genes, showing that a larger portion of the population experiences some climbing and useful movement. The greatest impact is found in 12 genes, where the mean grows significantly over generations, indicating that a larger genome offers enough space to evolve improved contact-based climbing rather than random motion.

4.2.2 Genome Size vs Best Fitness

The best-fitness plot improves with increasing genome size. 12 genes perform best and reach the maximum best fitness.



Img 7: Graph for the genome size vs best fitness

The best-fitness graph shows that extra genes increase the chance of finding an excellent climber, as seen by the fact that 3 genes give only moderate best scores, 5 genes improve slightly, and 12 genes reach far greater peaks.

4.2.3 Table Summary

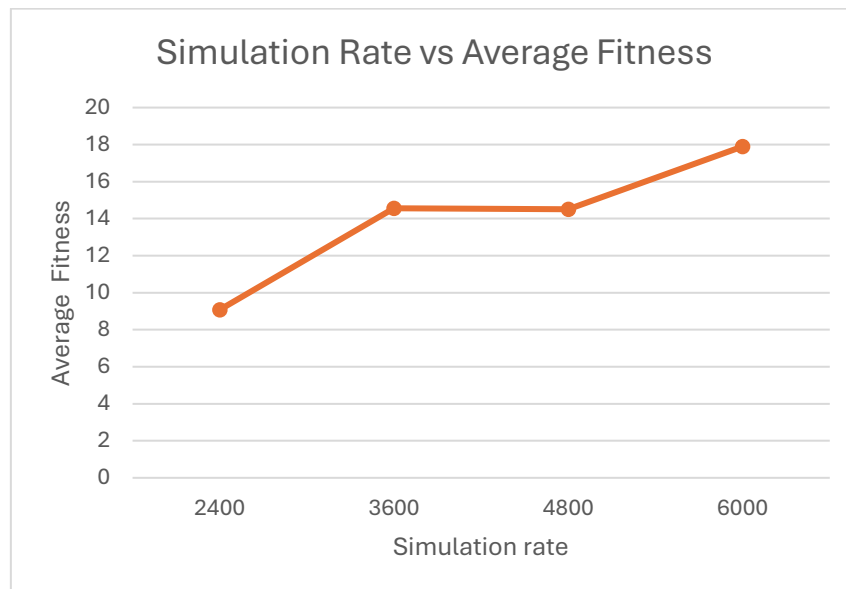
Run ID	Population Size	Initial Genome Size	Mutation Rate	No. of Generation	Simulation Rate	Best Fitness (Highest)	Average mean Fitness (Out of 3 Runs)	Average Best Fitness (Out of 3 Runs)
1	10	3 genes	0.1	300	2400	149.323	9.078	83.534
2	10	5 genes	0.1	300	2400	223.104	17.888	153.863
3	10	8 genes	0.1	300	2400	648.793	17.190	269.290
4	10	12 genes	0.1	300	2400	1058.465	53.023	688.931

Table 2: Genome Size

Larger genomes improve the possibility of finding at least one high-performing climber, as shown by the Average Best Fitness increasing progressively from 83.534 to 688.931 although the Average Mean Fitness is identical for 5 of 17.888 and 8 genes of 17.190. Increased overall variation at 8 genes is shown by the tiny decrease in average mean between 5 and 8 genes. This is expected since growing genome size also expands the search space. 12 genes are the optimal setting within the tested range because it improves both the population average and the best solutions found.

4.3 Simulation rate

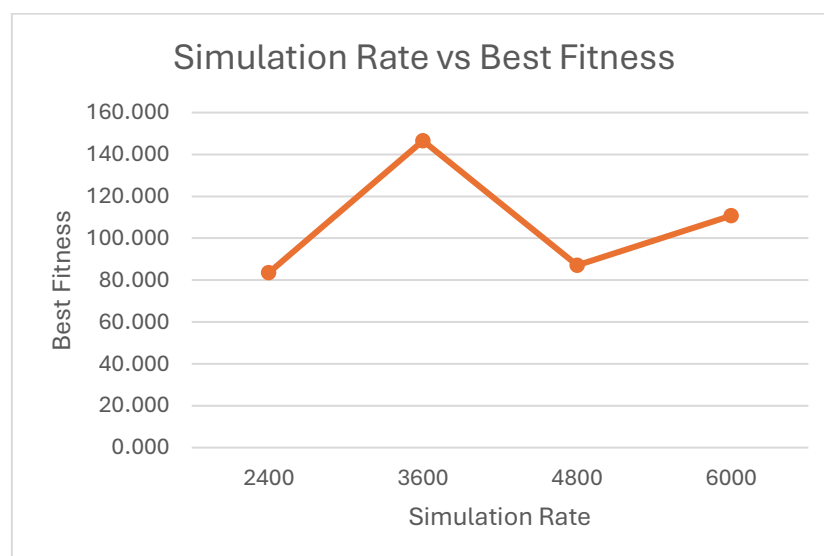
4.3.1 Simulation Rate vs Average Fitness



Img 8: Graph for simulation rate vs average fitness

Increasing the simulation rate improves the fitness. Since fewer creatures show consistent movement on the mountain, the 2400 setting remains the lowest. The increase to 3600 improves overall learning by raising the mean early and maintaining it higher. By the end, 6000 gives the highest mean, showing that a faster simulation rate could help the average population behave more consistently. At 4800, the mean trend slightly decreases from 3600.

4.3.2 Simulation Rate vs Best Fitness



Img 9: Graph for simulation rate vs best fitness

The strongest creature is not usually found at the greatest simulation rate, according to the best-fitness curve. This option became the most successful climber, as shown by the best outcome at 3600, which reaches the highest peak. However, 4800 and 6000 are below the baseline best score, indicating that even though they improve average behaviour, extremely high simulation rates could decrease the chance of finding a good creature.

4.3.3 Table Summary

Run ID	Population Size	Initial Genome Size	Mutation Rate	No. of Generation	Simulation Rate	Best Fitness (Highest)	Average mean Fitness (Out of 3 Runs)	Average Best Fitness (Out of 3 Runs)
1	10	3 genes	0.1	300	2400	149.323	9.078	83.534
2	10	3 genes	0.1	300	3600	303.425	14.565	146.552
3	10	3 genes	0.1	300	4800	147.399	14.510	87.109
4	10	3 genes	0.1	300	6000	146.204	17.898	110.846

Table 3: Simulation rate

In terms of creating good creatures across runs, 6000 has the best mean fitness of 17.898, but 3600 has the best single climber of 303.425 and the highest Average Best Fitness of 146.552, making it the strongest setting overall.

5. Experimenting with the encoding scheme

5.1 Baseline

Baseline uses the original encoding, allowing the genome to change both motor control settings and body form. In addition to giving the largest search space, this serves as a reference for the two advanced encoding tests.

5.2 Motor-Only

This experiment starts with a fixed robot design and just changes the motor control parameters. The goal was to find out whether control optimisation alone could improve climbing performance while removing changes in shape from the search space.

```
# if mode is motor only or motor only blend, start with the same elite body
if EXPERIMENT in ["motor_only", "motor_only_blend"]:
    path = "archive/Fitness_baseline/2026-01-01_20-41-50/elite_250.csv"
    # if the path exists
    if os.path.exists(path):
        basebody = genome.Genome.from_csv(path)
        # Give all creatures the same starting body
        for cr in pop.creatures:
            cr.update_dna(basebody)
```

Code snippet 11: Include motor-only and motor_only_blend version (ga.py)

```

# point_mutate changed for advanced part to include the motors
@staticmethod
def point_mutate(genome, rate, amount, motor_only=False):
    # mutate gene values with random change
    new_genome = copy.copy(genome)
    motorindex = None
    if motor_only:
        # get gene spec
        spec = Genome.get_gene_spec()
        # indexes of motor genes
        motorindex = {
            spec["control-waveform"]["ind"],
            spec["control-amp"]["ind"],
            spec["control-freq"]["ind"],
        }
    for gene in new_genome:
        for i in range(len(gene)):
            # skip body genes when motor_only is true
            if motorindex is not None and i not in motorindex:
                continue
            # if random more than rate
            if random.random() < rate:
                # create random change and add to gene value
                randomchange = (random.random() - 0.5) * 2 * amount
                gene[i] += randomchange
            # values stay between 0 and 1
            if gene[i] >= 1.0:
                gene[i] = 0.9999
            if gene[i] < 0.0:
                gene[i] = 0.0
    # return the new genome
    return new_genome

```

Code snippet 12: *point_mutate* changed to include the motor genes (*genome.py*)

So, only the motor-related genes were allowed to change, while the body genes were just frozen. Because the creature cannot change its structure to improve its grip or stability on the hill, this limits the quality of the solution while also making evolution more focused and quicker to explore.

5.3 Motor-Blended

The motor-blended extends Motor-only by changing the motor encoding to enable a smoother range of control signals instead of just one kind of waveform.

```

# get_output changed to include the blend version for the advanced part
def get_output(self):
    # move it forward and close around
    self.phase = (self.phase + self.freq) % (np.pi * 2)
    # If not blend
    if not self.blend:
        # if pulse type
        if self.motor_type == MotorType.PULSE:
            # +1 for first hald, -1 for second
            output = 1 if self.phase < np.pi else -1
        else:
            # sine output
            output = np.sin(self.phase)
        # multiple by amp which is how strong the motor is
        return self.amp * output

    # blend amount
    wave_form = float(self.waveform)
    # pulse value
    pulse = 1 if self.phase < np.pi else -1
    # sine value
    sine = np.sin(self.phase)
    # mix value
    sin_pulsemixed = (1.0 - wave_form) * pulse + wave_form * sine
    # multiple by amp which is how strong the motor is
    return self.amp * sin_pulsemixed

```

Code snippet 13: `get_output` function changed to include the blend (`creature.py`)

```

def get_motors(self, blend=False):
    # get expanded links
    self.get_expanded_links()
    # create motors
    if self.motors is None:
        motors = []
        # create one motor per joint
        for i in range(1, len(self.exp_links)):
            l = self.exp_links[i]
            # motor use link control values using blend
            m = Motor(l.control_waveform, l.control_amp, l.control_freq, blend=blend)
            motors.append(m)
        self.motors = motors
    return self.motors

```

Code snippet 14: `get_motors` changed to include the blend (`creature.py`)

```
def update_motors(self, cid, cr):
    """
    cid is the id in the physics engine
    cr is a creature object
    """
    # loop all joint and apply force and velocity
    for jid in range(p.getNumJoints(cid, physicsClientId=self.physicsClientId)):
        m = cr.get_motors(blend=self.motor_blend)[jid]
        p.setJointMotorControl2(cid, jid,
                                controlMode=p.VELOCITY_CONTROL,
                                targetVelocity=m.get_output(),
                                force = 50,
                                physicsClientId=self.physicsClientId)
```

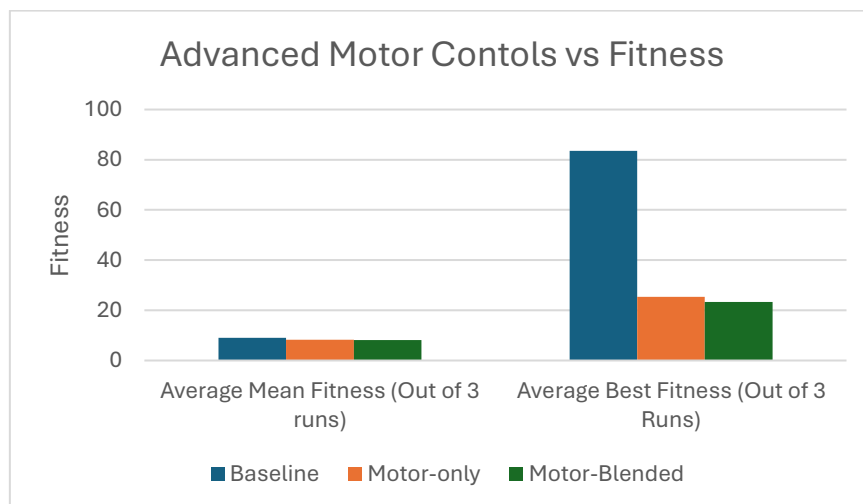
Code snippet 15: `update_motors` changed to include the blend (simulation.py)

A waveform parameter is used as a blend weight between pulse and sine, as instead of selecting only one. The goal is to improve control flexibility without changing the structure of the body. In contrast to evolving whole structure, this checks if greater motor encoding alone can restore performance.

5.4 Results

5.4.1 Advanced Motor Controls vs Fitness

The baseline encoding beats both motor-only changes in terms of learning, according to the fitness curves. Because baseline can improve both structure and control together, it rises to higher best fitness.



Img 10: Graph for different motor controls

This is important when mountain climbing, where shape plays a major role in grip, stability, and contact. Motor-Only and Motor-Blended, on the other hand, barely improve and plateau early, suggesting that control tuning is insufficient for climbing under the anti-cheat restrictions.

5.4.2 Table Summary

Versions	Population Size	Initial Genome Size	Mutation Rate	No. of Generation	Simulation Rate	Best Fitness (Highest)	Average mean Fitness (Out of 3 Runs)	Average Best Fitness (Out of 3 Runs)
Baseline	10	3 genes	0.1	300	2400	149.323	9.078	83.534
Motor-only	10	3 genes	0.1	300	2400	29.291	8.322	25.313
Motor-Blended	10	3 genes	0.1	300	2400	23.399	8.131	23.308

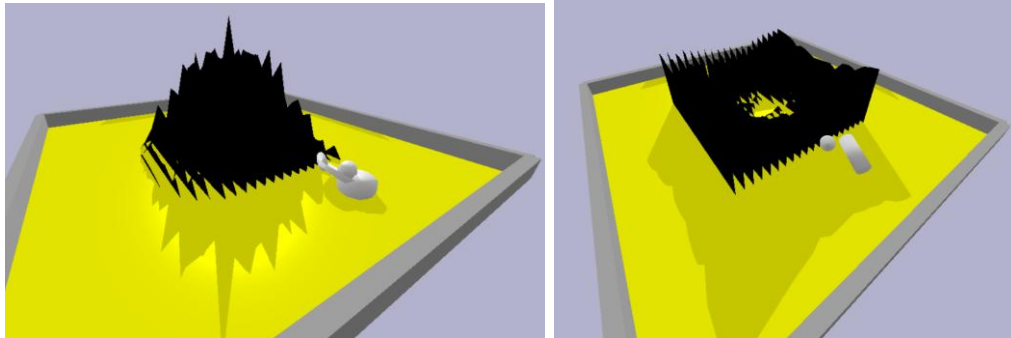
Table 4: Different motor controls

The best results are achieved by Baseline Best Fitness 149.323, Average mean 9.078, Average Best 83.534. Motor-Blended performs slightly worse Best 23.399, Average mean 8.131, Average Best 23.308, while Motor-Only performs far less well Best 29.291, Average mean 8.322, Average Best 25.313. Findings show that improving motor control without changing the body is too restrictive for this task. Effective climbing is mostly dependent on structural changes.

6. Exceptional Grade

6.1 Experiment with different landscapes

Two more landscapes Hills and Valleys were looked at to see if the evolved behaviour goes beyond a single terrain shape.

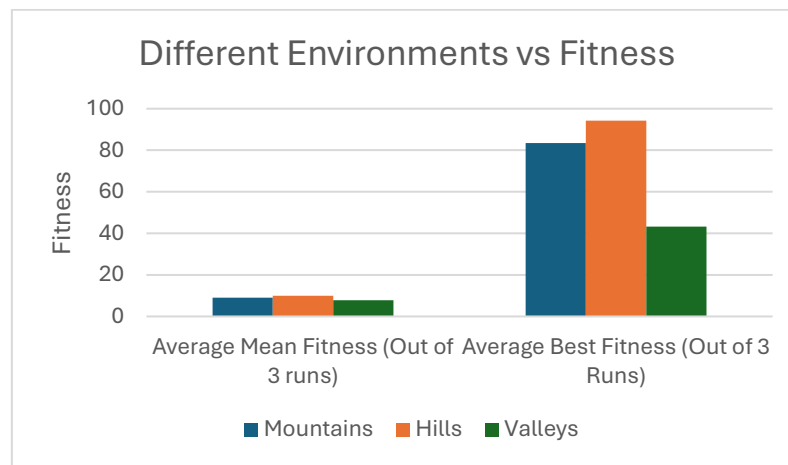


Img 12: pybullet loading mountain and valleys

These settings change local slopes, which influences grip, stability, and the viability of a climbing technique. To ensure a fair comparison, the terrain URDF was switched, but the GA settings, spawn conditions, fitness function, and anti-cheat regulations remained the same. This experiment is helpful because strong movement should stay competitive across all terrain shapes, while a creature that excels just on the Gaussian pyramid may be overfitted to that certain shape.

6.2 Results

6.2.1 Different Environments vs Fitness



Img 13: Graph for different landscapes

The performance on the hills is higher than the baseline for the mountains in all major fitness, while performance on the valleys is still far lower. This suggests that when the slope is still generally climbable, the evolved climbing pattern behaves well. However, it decreases when the terrain geometry changes the local slope directions and stability requirements.

6.2.2 Table Summary

Versions	Population Size	Initial Genome Size	Mutation Rate	No. of Generation	Simulation Rate	Best Fitness (Highest)	Average mean Fitness (Out of 3 Runs)	Average Best Fitness (Out of 3 Runs)
Mountains	10	3 genes	0.1	300	2400	149.323	9.078	83.534
Hills	10	3 genes	0.1	300	2400	169.449	9.891	94.169
Valleys	10	3 genes	0.1	300	2400	55.222	7.747	43.241

Table 5: Different Landscapes

Hills achieves the best overall results of Best Fitness 169.449, Average Mean 9.891, Average Best 94.169, while Valleys sees a decrease of Best 55.222, Average Mean 7.747, Average Best 43.241. The results show limited robustness to valleys but partial generalisation for Mountains and Hills, suggesting that training across multiple terrain types rather than a single landscape shape would be necessary for stronger generalisation.

7. Conclusion

This project created creatures that demonstrate true contact-based mountain climbing under anti-cheat rules. To reward approach, slope engagement, and consistent uphill progress, a specific fitness function was made. The findings shows that performance is affected by parameters like genome size and simulation speed,

and creating both body and control performs better than motor-only encoding. Multiple types of training would be required for better generalisation because testing on hills and valleys also showed that performance depends on terrain shape.